

데이터베이스 설계 | Database Design

MongoDB 스키마 설계와 데이터 모델링

문서 모델링 원칙부터 집계 파이프라인, 인덱스 최적화까지

강의 목차 (총 6개 단원)

Part 1: NoSQL & MongoDB 기초

- NoSQL의 등장 배경과 분산 시스템
- CAP 정리와 MongoDB의 설계 철학
- JSON과 BSON 데이터 구조 비교

Part 2: 데이터 모델링 원칙

- 유연한 스키마와 스키마 검증
- 임베딩(Embedding)과 참조 (Referencing)
- 관계 차수(1:1, 1:N, N:M)별 설계 패턴

Part 3: 고급 스키마 설계 패턴

- 버킷 패턴과 계산 패턴
- 부분집합 패턴과 다형성 패턴
- 이커머스 다중 필터를 위한 속성 패턴

Part 4: 백엔드 통합 및 Mongoose

- Mongoose ODM의 개념과 스키마 정의
- 미들웨어 훅(Middleware Hook) 활용
- 데이터베이스 인덱싱 (Indexing) 최적화

Part 5: 집계 파이프라인

- \$match, \$group, \$project 스테이지
- \$lookup을 활용한 컬렉션 조인
- Explain Plan을 이용한 쿼리 성능 분석

Part 6: 블로그 설계 실습

- 블로그 시스템 요구사항 분석
- 대량 댓글 페이징과 양방향 참조
- 동시성 이슈를 고려한 추천 기능 설계

PART 1 NoSQL & MongoDB 기초

기본 동작 원리와 데이터 구조의 이해

웹 서비스 아키텍처 변화와 NoSQL의 도입 배경

- 비정형 데이터의 증가: 로그, 센서 수집 값, 소셜 미디어 피드 등 형태가 고정되지 않은 대용량 데이터의 적재와 빠른 처리가 필요해졌습니다.
- 유연한 데이터 구조 요구: 서비스 배포 주기가 빨라짐에 따라, 관계형 데이터베이스(RDBMS)의 스키마 변경(DDL 실행)에 수반되는 리스크와 비용을 완화하고자 했습니다.
- 수평 확장성(Scale-out): RDBMS는 주

RDBMS와 MongoDB의 설계 철학 비교

RDBMS (MySQL, PostgreSQL 등)

- Database →
Table → Row /
Column
- 데이터 정규화
(Normalization)
를 통한 중복성
최소화
- 외래키(Foreign
Key)와 JOIN을
사용해 데이터를
관계 매핑
- 여러 행과 테이블을 포함하는
트랜잭션 처리에
익숙한 모델

MongoDB

- Database →
Collection →
Document / Field
- 데이터 비정규화
(Denormalization)
를 통한 읽기 성능
우선
- 함께 읽히고 크기가
제한되는 연관
데이터는 문서에
내장하는 방식을
먼저 검토
- 단일 문서 내의 원
자성(Atomicity)을
보장하며, 필요시
다중 문서 트랜잭
션 사용

분산 시스템 과 CAP 정리

- 분산 시스템에서 세 보장을 동시에 만족하기 어렵다는 이론입니다.
- C (Consistency, 일관성): 어떤 노드에 요청하든 가장 최근에 쓰인 동일한 데이터를 반환합니다.
- A (Availability, 가용성): 일부 노드 장애 시에도 살아있는 노드는 항상 정상 응답합니다.
- P (Partition Tolerance, 분할 내성): 노드 간 네트워크 단절(분할)이 발생해도 시스템이 유지됩니다.
- Replica Set: Primary 노드 중심의 쓰기 작업과 세컨더리 복제를 활용합니다.
- 네트워크 분할 시
readConcern / writeConcern
설정에 따라 합의 수준이 결정됩니다.

네트워크 장애 발생 시 데이터베이스의 선택

CA (가용성+일관성):
단일 노드 RDBMS (분
산 네트워크 고려 없
음)

AP (가용성+분할내
성): Cassandra,
DynamoDB (최종 일
관성 보장)

CP 성향 시스템:
HBase, primary 기반
replica set 구성의
MongoDB

CAP 분류는 암기보다 장애 상황에서 어떤 보장을 선택하는지 이해하는 것이 중요합니다.

BSON(Binary JSON)의 특징 과 구조

- MongoDB는 네트워크 전송 및 디스크 저장 시 JSON이 아닌 BSON 형식을 사용합니다.
- JSON의 구조적 한계: 텍스트 형식이므로 파싱 비용이 높고, 날짜(Date)나 상세 숫자(Int32, Decimal128 등) 타입을 기본적으로 구별하지 못합니다.
- BSON의 설계상 장점:
 - 바이너리 형태로 직렬화되어 인코딩/디코딩 성능이 우수합니다.
 - 필드 길이와 요소의 위치가 저장되므로, 쿼리 수행시 필요한 필드만 건너뛰며 스캔할 수 있습니다.
 - 날짜와 정밀한 숫자 연산을 위한 추가 타입을 내장

```
// BSON 명세에 따른 데이터  
표현 예시  
{  
  "_id":  
  ObjectId("507f191e810c197  
  "title": "BSON 구조의  
  이해",  
  "createdAt":  
  ISODate("2026-05-  
  27T10:00:00Z"),  
  "views":  
  NumberInt(1500),  
  "price":  
  NumberDecimal("19.99"),  
  "is_active": true  
}
```

문서(Document) 모델의 중첩 구조 표현

- 관계형 데이터베이스에서는 다중 값을 저장할 때 데이터의 무결성을 위해 별도 테이블을 생성하고 JOIN 처리를 요구하는 것이 불편적입니다.
- MongoDB는 배열(Array)과 중첩 문서(Embedded Document)를 네이티브 데이터 타입으로 지원합니다.
- 애플리케이션 계층의 객체 모델을 영속성 계층에 직접 투영할 수 있어, 임피던스 미스매치(Impedance Mismatch) 현상을 줄여줍니다.

설계 권장 조건:

자주 같이 조회되고 갱신되는 관련 데이터 그룹은 데이

PART 1 | NoSQL & MongoDB 기초 MongoDB 기본 CRUD 예제

- MongoDB는 collection 단위로 document를 추가, 조회, 수정, 삭제합니다.
- SQL의 행(row)보다 중첩 객체와 배열을 자연스럽게 다루는 방식에 가깝습니다.
- 강의용 예제에서는 MongoDB shell 문법을 사용합니다.

```
// MongoDB shell 예제
db.users.insertOne({
  name: "Alice",
  email:
    "alice@example.com",
  roles: ["student"],
  createdAt: new Date()
});

db.users.find(
  { roles: "student" },
  { name: 1, email: 1, _id:
    0 }
);

db.users.updateOne(
  { email:
    "alice@example.com" },
  { $set: { lastLoginAt: new
    Date() } }
);

db.users.deleteOne({ email:
  "alice@example.com" });
```

Projection: 필요한 필드만 읽기

- MongoDB 쿼리는 조건 뿐 아니라 반환 필드도 설계 대상입니다.
- 큰 문서에서 목록 화면에 필요 없는 본문, 첨부 파일, 메타데이터를 제외할 수 있습니다.
- Projection은 네트워크 전송량과 애플리케이션 처리량을 줄이는 데 도움이 됩니다.

```
// MongoDB shell 예제
db.posts.find(
  { status:
    "published" },
  {
    title: 1,
    authorName: 1,
    publishedAt: 1,
    _id: 0
  }
);
```

PART 2 데이터 모델링 핵심 원칙

스키마리스 환경에서의 효율적인 관계 모델링과 정합성 관리

유연한 스키마와 데이터 검증(Schema Validation)

- MongoDB는 고정 스키마를 강제하지 않으므로 초기 요구사항 변경에 대응하기 쉽습니다.
- 다만, 스키마의 제약이 지나치게 느슨해질 경우 필드명 불일치나 누락으로 인한 데이터 분석 및 서비스 장애가 발생할 수 있습니다.
- 따라서 운영 단계로 전환할 때는 **Mongoose ODM의 스키마 제약 조건**을 설정하거나 데이터베이스 엔진 내에 **JSON Schema Validation** 정책을 도입하여 유효성을 보완합니다.



MongoDB JSON Schema Validation 예제

- Flexible schema
는 검증을 하지
않는다는 뜻이 아
닙니다.
- 운영 단계에서는
DB 레벨 검증 또
는 ODM 검증을
함께 검토합니다.
- 검증 규칙은 핵심
필드부터 작게 시
작하는 편이 관리
하기 쉽습니다.

```
// MongoDB shell 예제
db.createCollection("u
{
  validator: {
    $jsonSchema: {
      bsonType:
"object",
      required:
["email", "name"],
      properties: {
        email: {
          bsonType:
"string",
          pattern:
"^.+@.+\\.\\.\\.\\.+.$"
        },
        name: {
          bsonType: "string"
        },
        age: {
          bsonType: "int",
          minimum: 0 }
        }
      }
    }
  });
```


임베딩(Embedding)과 참조(Referencing)의 선택 기준

임베딩(Embedding)

- 관련 데이터를 단일 문서 내에 하위 요소로 배치합니다.
- **장점:** 데이터의 참조 비용이 없어 읽기 속도가 빠릅니다.
- **단점:** 16MB 크기 제한에 걸릴 위험이 있으며, 비정규화로 인한 갱신 비용이 큼니다.

참조(Referencing)

- 독립된 컬렉션에 저장한 뒤 `_id` 식별자로 조회합니다.
- **장점:** 정규화되어 데이터 정합성 관리가 쉽고, 문서 확장 한계를 방지합니다.
- **단점:** 데이터 통합 시 조인 연산 (`$lookup`) 또는 N+1 조회 쿼리가 수반됩니다.

판단 기준 예제: 사용자와 프로필

- 함께 조회되고 함께 갱신되는 profile은 임베딩을 검토합니다.
- 계속 증가하고 독립적으로 조회되는 activity log는 참조 구조가 더 적합합니다.
- 문서 경계는 화면, API, 갱신 단위를 함께 보고 정합니다.

```
// 함께 조회되는 프로필:
document 예제
{
  _id:
  ObjectId("66500000000000000000")
  name: "Alice",
  profile: {
    school: "JBNU",
    github:
    "github.com/alice"
  }
}

// 계속 증가하는 활동 로그: 별도
컬렉션
{
  user_id:
  ObjectId("66500000000000000000")
  type: "login",
  createdAt: ISODate("2026-
05-27T10:00:00Z")
}
```

PART 2 | 데이터 모델링 원칙 1:1 관계 설계 패턴

- 두 엔티티가 라이프사이클을 공유하며 상호 일대일로 맵핑되는 도메인 모델입니다.
- 권장 방안: 데이터의 보안 등급 차이나 특정 필드의 용량(예: 바이너리 첨부 파일)이 문제되지 않는다면 임베딩 처리를 우선합니다.
- 한 차례의 디스크 I/O 작업만으로 필요한 리소스를 수집하기 때문에 비효율적인 조인 작업을 줄일 수 있습니다.

```
{
  "_id": "user_001",
  "name": "이경수",
  "profile": { // 1:1 결합 형태의 임베
    "age": 25,
    "address": "전라북도 전주시 백제대로",
    "github": "github.com/macslab"
  }
}
```


1:N 관계: 데이터 증가 규모에 따른 설계

- 관계형 모델은 외래키 정규화로 일괄 처리하지만, 문서형 데이터베이스는 관계의 성장 크기에 따라 다른 접근이 요구됩니다.
- **One-to-Few (소규모 관계):** 데이터가 많아야 수십 개 미만인 경우. 하나의 부모 문서 내부에 임베딩 배열로 축적합니다. (예: 사용자의 배송지 목록)
- **One-to-Many (중규모 관계):** 데이터가 수백~수천 개로 유한한 경우. 독립된 컬렉션으로 구현하되 부모 문서에서 자식 문서의 식별자 배열을 유지할 수 있습니다. (예: 카테고리-상품 리스트)
- **One-to-Squillions (대규모 관계):** 데이터가 끊임없이 증가하고 수만 건 이상 늘어날 수 있는 경우. 부모의 식별자 배열이 16MB 상한선을 초과할 우려가 있으므로, 자식 문서 쪽에서 부모의 `id` 를 참조하

1:N 관계: 적은 수의 관계 (임베딩)

- 사용자의 복수 연락처나 거주지 정보 등 추가되는 데이터 개수가 예측 가능하고 한정적인 관계입니다.
- 배열의 증가폭이 제한적이므로 문서가 비대해져 성능 저하나 한계를 유발할 여지가 상대적으로 적습니다.
- 조회 과정을 단순하게 만들고 단일 문서 쓰기의 원자성을 활용하기에 유용합니다.

```
{  
  "name": "Alice",  
  "addresses": [  
    { "type": "home",  
      "city": "Jeonju" },  
    { "type": "work",  
      "city": "Seoul" }  
  ]  
}
```

1:N 관계: 대규모 관계 (부모 참조)

- 특정 장비의 모니터링 로그나 결제 트랜잭션 기록처럼 수시로 추가되며 양이 매우 많은 도메인 관계입니다.
- 부모 문서(Host)에 자식 데이터 배열을 포함하는 경우, 얼마 지나지 않아 BSON 문서 용량 제한인 16MB를 충족하지 못해 에러가 유발됩니다.
- 따라서 각 로그 문서가 부모 장비 문서의 식별자를 저장하게 함으로써 무제한적 성장을 회피합니다.

```
// Host Collection
{ "_id": "server_1",
  "ip": "192.168.0.1" }
```

```
// Logs Collection (부모 ID 참조)
{
  "host_id":
    "server_1",
  "msg": "CPU
Overload",
  "time":
    ISODate("2026-05-
27T10:00:00Z")
}
```

N:M 관계: 다대다 관계 설계

- 학습자와 교과목처럼 상호 다대다 구조를 이루는 테이블 매핑 구조입니다.
- 관계형 스키마와 달리 연결 관계를 중간에서 매개하는 매핑 테이블 없이, 도메인 문서에 식별자 배열 필드를 직접 보유하는 형태로 설계 가능합니다.
- 양방향 참조(Two-way Referencing): 양 개체의 성장 규모가 16MB 한도를 넘지 않는 조건 하에, 도메인 문서에 서로의 식별자 배열을 유지하여 양측 검색 속도를 개선합니다.

```
// Student Document  
(수강 목록 참조)  
{  
  "name": "Alice",  
  
  "enrolled_courses":  
  ["CS101", "DB202"]  
}
```

```
// Course Document  
(수강생 목록 참조)  
{  
  "_id": "DB202",  
  "students":  
  ["Alice_ID",  
   "Bob_ID"]  
}
```

PART 3 고급 스키마 설계 패턴

비정규화 특성을 활용한 데이터 조회 및 저장성 개선 기법

PART 3 | 고급 스키마 설계 패턴 임베딩 패턴 (Embedding Pattern)

- 주문 상세처럼 항상 함께 읽히는 데이터는 한 문서에 모아 두면 조회 과정이 단순해집니다.
- 관련 데이터의 개수와 크기가 제한적일 때 임베딩을 먼저 검토합니다.
- 여러 문서에서 공유하는 값이 자주 바뀐다면 중복 저장 때문에 갱신 비용이 커질 수 있습니다.

```
// 배송 정보와 항목을 포함하는 단일 주문 문서
{
  "order_id": "ORD-1234",
  "total_price": 55000,

  "shipping_address": {
    "zip": "12345",
    "street": "Seoul"
  },
  "items": [
    {"prod_id": 1, "qty": 2},
    {"prod_id": 5, "qty": 1}
  ]
}
```

참조 패턴 (Reference Pattern)

- 공통 데이터를 한 곳에서 관리해야 하거나 문서가 지나치게 커지는 상황에서는 참조 구조를 검토합니다.
- 도메인 개체들이 각각 자주 조회되거나, 부모-자식 관계의 크기가 커 16MB 제한을 넘기 쉬울 때 활용합니다.
- 참조 구조를 사용하면 JOIN 비용과 추가 쿼리 비용이 생기므로 조회 패턴을 함께 확인해야 합니다.

```
// 출판사 정보 (독립  
컬렉션)  
{ "_id": "pub1",  
  "name": "O'Reilly"  
}
```

```
// 도서 정보 (식별자로  
출판사 참조)  
{  
  "title":  
    "MongoDB The  
    Definitive Guide",  
  "publisher_id":  
    "pub1"  
}
```

버킷 패턴 (Bucket Pattern): 시계열 데이터 최적화

- 해결할 문제: IoT 기기, 주식 거래 틱 데이터 같은 연속적 데이터 적재 시, 매번 별도 문서를 삽입하면 인덱스 오버헤드와 저장 공간 메타데이터 중복이 커집니다.
- 설계 방식: 일정한 기준 범위(예: 1일 단위 혹은 수집 1,000건 단위)별로 문서를 하나 생성하고, 그 내부 배열에 측정값을 차례대로 push하여 적합한 버킷을 구축합니다.

PART 3 | 고급 스키마 설계 패턴 버킷 패턴: 측정값 추가 예제

- 버킷은 시간 범위나 개수 기준으로 문서를 묶어 쓰기 부담을 줄이는 패턴입니다.
- `upsert` 를 사용하면 해당 시간 범위의 버킷이 없을 때 새로 만들 수 있습니다.
- 버킷 크기와 시간 범위를 정하지 않으면 한 문서가 계속 커질 수 있습니다.

```
// MongoDB shell 예제
db.sensor_buckets.updateOne(
  {
    sensor: "Temp_01",
    startDate: ISODate("2026-05-27T00:00:00Z")
  },
  {
    $push: {
      measurements: {
        t: ISODate("2026-05-27T10:01:00Z"),
        value: 24.6
      }
    },
    $set: { endDate:
ISODate("2026-05-27T10:01:00Z") }
  },
  { upsert: true }
);
```

계산 패턴 (Computed Pattern): 사전 집계 연산

예를 들어, 대용량 데이터 세트에서 모든 트래픽에서 게시글의 누적 조회 수, 영화 평점 평균 등을 조회할 때마다 실시간 계산을 수행하면 읽기 비용이 커질 수 있습니다.

- 처리 방식: 쓰기 시점의 비용을 조금 높더라도, 연계 결과 데이터를 미리 집계하여 문서의 지정 필드에 기록해 둡니다.
- 원자적 증감 연산자(`$inc`) 등을 동원하여 동시성 문제를 제어하며 통계 정보를 갱신

통계 필드 상시 갱신

평점 등록 시마다 전체 연산(Average)을 배제하고 부분 누적 처리 적용

```
{
  "title":
    "Inception"

  "total_revi
    15000,
    // 사전 계
    산값

  "total_scor
    72000,
    // 사전 계
    산값

  "avg_rating
    4.8
    //
    (total_scor
    /
    total_revie
  }
```

PART 3 | 고급 스키마 설계 패턴

계산 패턴: 평점 누적 업데이트

- 조회할 때마다 평균을 다시 계산하지 않고 누적값을 함께 갱신할 수 있습니다.
- 평균값을 저장할지, 합계와 개수만 저장할지는 조회 요구와 정합성 기준에 따라 정합니다.
- 단일 문서 update는 원자적으로 처리됩니다.

```
// MongoDB shell 예제
db.movies.updateOne(
  { _id: movieId },
  {
    $inc: {
      total_reviews: 1,
      total_score: 5
    },
    $set: {
      updatedAt: new Date()
    }
  }
);
```

부분집합 패턴 (Subset Pattern): 최신 데이터 우선 노출

- 주요 배경: 메모리(RAM)에 활성 상태로 적재되는 데이터셋 영역인 Working Set의 부피가 커지면 디스크 접근이 늘어 응답 속도가 떨어질 수 있습니다.
- 설계 방식: 전체 연계 데이터 중 자주 쓰는 최신 데이터셋 혹은 상위 항목(예: 최근 리뷰 10개)만 부모 문서에 배치하고, 나머지 데이터는 별도 컬렉션에 참조 형태로 보관합니다.
- 기본 조회는 가볍게 유지하면서 전체 히스토리 조회 요구도 처리할 수 있습니다.

```
// 상품 문서 (최근 작성 5건 리뷰 내장)
{
  "name":
  "iPhone 17",
  "recent_reviews"
  [
    { "text": "배
```



```
// 전체 리뷰 보관용 컬렉션 (더보기 요구 시 조회)
{
  "prod_id":
  "iPhone 17",
  "text": "배
  손이 아쉽습니
```

부분집합 패턴: 최근 댓글 캐시 갱신

- 부모 문서에는 최근 댓글 일부만 유지하고 전체 댓글은 별도 컬렉션에 보관합니다.
- `$slice` 로 부모 문서에 남길 최근 데이터 수를 제한합니다.
- 목록 첫 화면은 빠르게 보여주고, 더보기는 comments 컬렉션에서 조회합니다.

```
// MongoDB shell 예제
db.posts.updateOne(
  { _id: postId },
  {
    $push: {
      recent_comments: {
        $each: [{
          comment_id:
commentId,
          author: "Alice",
          text: "좋은 글입니
다.",
          createdAt: new
Date()
        }],
        $sort: { createdAt:
-1 },
        $slice: 5
      }
    }
  }
);
```


다형성 패턴 (Polymorphic Pattern)

- 주요 배경: 게시글 양식의 다형성이나 정기 결제 수단(신용카드, 간편결제, 휴대폰 결제)처럼 기본 정보는 공통되지만 하위 필드 구조가 다를 때 발생합니다.
- 관계형 데이터베이스에서는 각각의 하위 테이블을 분할하여 관계를 매핑한 뒤 JOIN 구조를 쓰게 되므로 스케일이 비효율적입니다.
- 해결 방식: 스키마의 유연성을 활용하여 단일 컬렉션에 다양한 양식의 문서를 직접 적재하고, 문서를 구별할 수 있는 식별 정보 필드(예: `type`)를 할당하여 통일된 형태로 읽습니다.

```
// 하나의 피드 컬렉션에 다양한 형식의 문서 적재
[
  { "type": "text", "content": "공지사항 내용"
},
```

속성 패턴 (Attribute Pattern): 동 적 필드 검색 인덱싱

- 핵심 배경: 의류의 사이즈/색상, 노트북의 RAM 용량/CPU 기종 등 상품 카테고리마다 검색이나 필터링 기준이 되는 특성이 가변적일 때 유용합니다.
- 개별 특성을 단독 필드로 설계하면 인덱스가 많아지고 관리 비용이 커질 수 있습니다.
- 처리 방식: 속성 명칭(Key)과 속성값(Value)을 하나의 쌍으로 묶어 배열 형태로 담고,
`{"specs.k": 1,`
`"specs.v": 1}` 복합 인덱스로 관리합니다.

개별 필드 설계 (인덱스 유지보수 난해)

```
{ "cpu": "M3", "ram":  
  "128GB" }
```

속성 패턴 적용 (복합 인덱스 검토)

```
{  
  "specs": [  
    { "k": "cpu", "v":  
      "M3" },  
    { "k": "ram", "v":  
      "128GB" }  
  ]  
}  
//  
db.products.createIndex(  
  1, "specs.v": 1})
```

PART 3 | 고급 스키마 설계 패턴 속성 패턴: 필터 검색 쿼리

- 상품 속성을 `specs: [{ k, v }]` 형태로 저장하면 카테고리별 가변 필터를 다루기 쉽습니다.
- `$elemMatch` 를 사용해야 같은 배열 원소 안의 `k / v` 조건을 묶을 수 있습니다.
- 필터 조합이 많을수록 실제 쿼리 패턴으로 인덱스를 확인해야 합니다.

```
// MongoDB shell 예제
db.products.createIndex({
  "specs.k": 1,
  "specs.v": 1
});

db.products.find({
  specs: {
    $elemMatch: {
      k: "ram",
      v: "16GB"
    }
  }
});
```


PART 3 | 고급 스키마 설계 패턴

설계 패턴 요약 및 선택 기준

패턴 명칭	설계상 고려 요인 및 이점	대표적인 적용 상황
Bucket (버킷)	시계열 데이터 메타데이터 제거 및 문서 수 조절	IoT 로그, 장비 지표 데이터 수집
Computed (계산)	반복 계산 성능 부하 분산 및 사전 캐싱	누적 평점 평균, 일일 정산 결과 기록
Subset (부분집합)	Working Set 메모리 오버헤드 완화 및 페이징 최적화	게시글 최신 댓글 목록, 최근 구매 이력
Polymorphic (다형성)	단일 컬렉션을 이용한 동적 스키마 쿼리 단일화	다양한 양식의 결제 수단, 소셜 뉴스 피드
Attribute (속성)	가변적 필터링 항목들에 대한 인덱스 복잡도 감축	가전제품 다차원 스펙 필터링 검색

PART
4

Node.js 백엔드 통합 및 Mongoose ODM

어플리케이션 통합과 인덱스를 활용한 조회 최적화

최신 MongoDB 기능 및 개발 동향



Vector Search
연계

텍스트의
미 기반
검색을 위
해 문장의
임베딩 벡
터와 원본
문서를 동
일 컬렉션
내에 융합
합니다.



버전
별
성능
개선
확인

버전
별로
쿼리
실행,
인덱
스, 샤
딩, 집
계 성
능이
개선
되므
로 운
영 환
경에
서는
릴리
즈 노
트와
호환
성을
함께



보안
암
호화
강
화

Queryable
Encryption
을 사용하
여 암호 상
태로 데이
터를 저장
하고 부분
조건 검색
을 지원합
니다.

Mongoose ODM의 역할과 필요성

1

스키마 및 데이터 타입 제약

문서 유입 전에 타입 에러 차단, 기본 값 설정 및 어플리케이션 계층의 제약 조건을 적용

2

참조 관계 해석의 편의성

`populate()`
메서드를 활용
해 다른 문서 컬
렉션을 조회한
뒤 객체 트리 구
조로 묶어줍니
다.

3

저장 전/후 처리

저장
작업
(pre-
save)
직전
의 암
호 갱
신 확
인이
나 삭
제 직
후의
후속
처리를 코
드로 관
리 합니
다.

Node.js에서 MongoDB 연결하기

- MongoDB URI는 코드에 직접 넣지 않고 환경변수로 관리합니다.
- 운영 환경에서는 connection error 처리와 로그 정책을 별도로 둡니다.
- 애플리케이션 시작 시점에 연결을 완료한 뒤 API 서버를 열도록 구성합니다.

```
// Node.js/Mongoose 예제
const mongoose =
  require("mongoose");

async function connectDB()
{
  await
  mongoose.connect(process.env.M
  {
    dbName: "lecture_blog"
  });

  console.log("MongoDB
  connected");
}

module.exports =
  connectDB;
```


Mongoose Schema 정의 예제

- User 스키마를 구성하여 엄격한 자료 형식 유효성을 체크하는 구조입니다.

```
// models/User.js (사용자 모델 스키마 정의)
const mongoose =
  require('mongoose');

const userSchema = new
  mongoose.Schema({
    username: { type: String,
      required: true, unique: true,
      minlength: 3 },
    email: { type: String, required:
      true, match: /.+@.+\..+/,
    password: { type: String,
      required: true }, // 비밀번호 해싱 연동
    role: { type: String, enum:
      ['user', 'admin'], default: 'user'
    },
    age: { type: Number, min: 0 },
  }, { timestamps: true });

module.exports =
  mongoose.model('User', userSchema);
```

Mongoose 쿼리 예제

- `select()` 로 필요한 필드만 가져옵니다.
- 읽기 전용 목록에서는 `lean()` 을 사용해 Mongoose document 변환 비용을 줄일 수 있습니다.
- 정렬과 제한 조건은 실제 인덱스와 함께 설계합니다.

```
// Node.js/Mongoose 예제
const posts = await
Post.find({
  status: "published",
  tags: "mongodb"
})
  .select("title
authorName publishedAt")
  .sort({ publishedAt: -1
})
  .limit(20)
  .lean();
```

Mongoose Middleware Hook 활용

- 사용자 문서를 저장하기 전에 비밀번호를 단방향 해시로 바꾸는 예제입니다.

```
const bcrypt =
  require('bcrypt');

// save 이벤트 처리 직전에 호출되는
pre-hook
userSchema.pre('save', async
function(next) {
  const user = this;

  // password 필드 값이 변경된 경우
  //에만 수행
  if
    (user.isModified('password'))
  {
    const salt = await
      bcrypt.genSalt(10);
    user.password = await
      bcrypt.hash(user.password,
        salt);
  }

  next();
});
```


관계 참조 처리를 위한 Populate

- 참조 구조(Referencing) 데이터 조회 시 수동으로 복수 조회를 하거나 `$lookup` 쿼리를 짜는 오버헤드를 최소화하기 위해 사용합니다.
- 스키마 상에 ref 옵션으로 연관 관계를 선언하고, 쿼리 수행 단계에서 `populate()` 를 명시하여 결합 데이터를 추출합니다.
- 내부적으로는 여러 번 조회하는 다수 쿼리로 나뉠 수 있으므로 루프 안에서 반복 호출하지 않도록 주의해야 합니다.

```
// Post Schema 설계 시 참조 설정
author: { type:
mongoose.Schema.Types.ObjectId,
ref: 'User' }

// -----

// 조회 시 체이닝 구문 연동
const posts = await
Post.find()
  .populate('author',
'username email'); // 필요한
타겟 필드 정의

// 결과: author 필드가 원본
User 문서의 정보로 주입됨
console.log(posts[0].author.us
```

인덱스(Index)의 기본 원리와 최적화

- COLLSCAN vs IXSCAN: 인덱스가 없으면 컬렉션 전체 문서를 물리적으로 모두 순차 탐색하므로 검색 속도가 선형으로 하락합니다. 인덱스는 검색 값을 B-Tree 인덱스 파일로 정렬하여 조회 경로를 줄여줍니다.
- 조회-쓰기 상호 절충 (Tradeoff): 인덱스는 데이터 읽기 성능을 크게 늘리는 대신, 쓰기/갱신 작업 시마다 인덱스 트리를 지속적으로 수정하므로 오버헤드가 누적됩니다.

의미론적 검색: Atlas Vector Search

더라도, 문장 의미의 코사인 유사도 연산으로 관련 있는 문서를 검색해 냅니다.

- MongoDB Atlas는 고차원 텍스트 임베딩 벡터값과 기존 일반 관계 메타데이터(예: 카테고리, 게시글 상태)를 단일 문서 스키마로 모아 하이브리드 형식으로 인덱싱할 수 있게 지원합니다.
- RAG(검색 증강 생성) 파이프라인에서 벡터 데이터 전용 DB 분리 없이

```
db.posts.aggregate([
  {
    $vectorSearch: {
      index:
        "post_embedding_index",
      path:
        "embedding",

      queryVector:
        userQuestionEmbedding,

      numCandidates:
        100,
      limit: 5,
      filter: {
        status:
          "published"
      }
    },
    {
      $project: {
        title: 1,
        summary: 1,
        score: {
          $meta:
            "vectorSearchScore"
        }
      }
    }
  ])
```

쿼리 성능 분석과 Explain 계획

- 데이터 성능 튜닝의 첫 단계는 쿼리의 실행 계획(Execution Plan)을 디버깅하여 비효율성을 제거하는 것입니다.
- 쿼리 메서드 후미에 `.explain("executionStats")` 를 체이닝하여 실행하면 상세 데이터 수치를 획득할 수 있습니다.
- 검토해야 할 주 지표:
 - `winningPlan.stage` : `IXSCAN` (인덱스 탐색) 여부를 확인하고, `COLLSCAN` (풀 스캔) 발생 여부를 파악합니다.
 - `executionTimeMillis` : 쿼리 실행에 가용한 물리적 경과 시간(ms)을 검토합니다.
 - `totalDocsExamined` : 쿼리 필터 처리를 위해 검사한 문서 개수입니다. 반환 결과 수와 가까울수록 이상적입니다.

운영 환경 점검:

대량 데이터 컬렉션에서 `COLLSCAN` 이 반복되면 디스크 I/O 대기과 서버 자원 소모로 이어질 수 있으므로 인덱싱 누락 여부를 먼저 확인해야 합니다.

Explain으로 인덱스 사용 확인하기

- `winningPlan` 에서 어떤 실행 계획이 선택되었는지 확인합니다.
- `totalDocsExamined` 가 결과 건수보다 과도하게 크면 인덱스 설계를 다시 봅니다.
- `executionTimeMillis` 는 테스트 데이터와 운영 데이터 차이를 고려해 해석합니다.

```
// MongoDB shell 예제
db.posts.find({
  status: "published",
  author_id: userId
})
.sort({ publishedAt: -1 })
.explain("executionStats");

// 확인 지표
// winningPlan,
// totalDocsExamined
// totalKeysExamined,
// executionTimeMillis
```

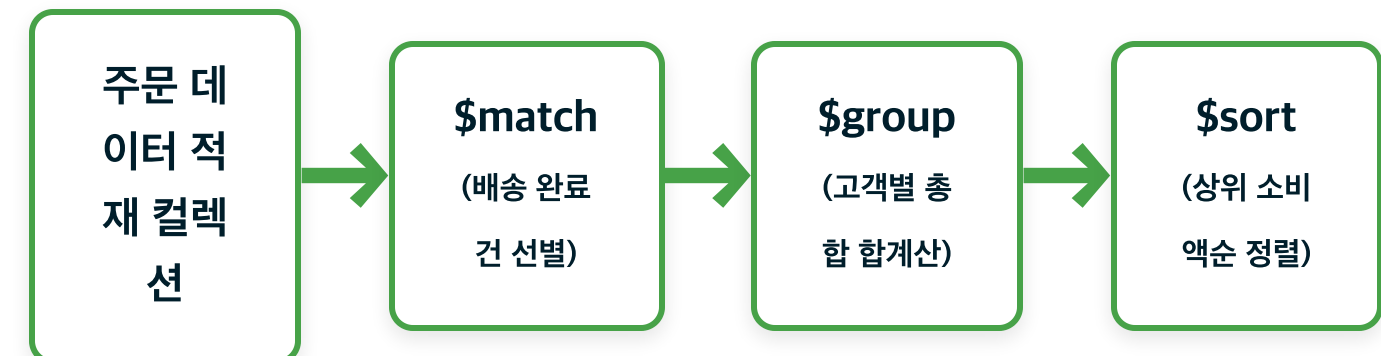

PART
5

집계 파이프라인 (Aggregation Pipeline)

다단계 가공(Pipeline)을 통한 데이터 분석 및 변환 기법

집계 파이프라인의 개념과 구조

- 동작 방식: 집계 파이프라인은 문서를 여러 단계에 통과시키면서 필터링, 그룹화, 변환을 차례로 수행합니다.
- Unix 터미널의 파이프 연동(`cmd1 | cmd2`)과 유사한 구조로 설계되어 있어, 앞선 스테이지의 아웃풋 스트림이 곧 다음 스테이지의 인풋 스트림으로 공급됩니다.
- SQL 언어의 `JOIN`, `GROUP BY`, `HAVING` 등에 대응되는 변환 연산을 애플리케이션으로 모두 가져오지 않고 MongoDB 안에서 처리합니다.



PART 5 | 집계 파이프라인

집계 파이프라인의 주요 스테이지

스테이지	SQL 대응어	기본 역할 및 가이드라인
\$match	WHERE	문서 필터링. 메모리 점유 및 연산 효율을 위해 가급적 맨 첫 단계에 배치합니다.
\$group	GROUP BY	지정 키 기준 그룹화. 개수 (sum), 평균(avg) 등의 누적 연산자를 수반합니다.
\$project	SELECT	출력 필드 지정. 신규 가공 필드 생성, 불필요 필드 억제 (0) 처리를 담당합니다.
\$sort / \$limit / \$skip	ORDER BY / LIMIT / OFFSET	메모리 정렬 및 노출 범위 필터링. 정렬 작업은 적합한 인덱스 지원이 필요합니다.
\$lookup	LEFT JOIN	이종 컬렉션 조인. 관계 구조를 복원할 때 조인 결과 배열을 문서에 맵핑합니다.

PART 5 | 집계 파이프라인 Aggregation 성능 점검 포인트

- `$match` 는 가능한 앞쪽에 두어 처리할 문서 수를 줄입니다.
- `$project` 로 큰 필드를 일찍 제거하면 이후 단계의 부담이 줄어듭니다.
- 큰 집계는 `allowDiskUse` 필요 여부와 정렬 인덱스를 함께 확인합니다.

```
// MongoDB shell 예제
db.orders.aggregate(
  [
    { $match: { status: "completed" } },
    {
      $project: {
        customer_id: 1,
        price: 1,
        date: 1
      }
    },
    {
      $group: {
        _id: "$customer_id",
        total: { $sum: "$price" }
      }
    },
    {
      $sort: {
        total: -1
      }
    }
  ],
  { allowDiskUse: true }
);
```

PART 5 | 집계 파이프라인 이커머스 주문 데이터 통계 분석 예제

- 집계 요구사항: 2026년 5월 완료
(completed)된 주문 중, 고객별 누적 주문 금액과 횟수를 구해 금액 순으로 상위 5명을 정렬하는 쿼리입니다.
- 단계별 스테이지를 배열 요소로 정렬하여 데이터베이스 엔진 내부에서 처리하도록 넘겨줍니다.

```
db.orders.aggregate([
  {
    $match: {
      status: "completed",
      date: {
        $gte: ISODate("2026-05-01"),
        $lt: ISODate("2026-06-01")
      }
    },
    {
      $group: {
        _id: "$customer_id",
        total_spent: { $sum: "$price" },
        order_count: { $sum: 1 }
      }
    },
    { $sort: { total_spent: -1 } },
    { $limit: 5 }
  ])
```

\$lookup 스테이지를 활용한 컬렉션 조인

- 역할: 관계형 데이터 베이스의 Outer Join 과 동등한 기법으로, 기준 문서와 대상 컬렉션 문서 간의 관계 매핑을 처리합니다.
- 조인 결과물은 명시한 식별 키(`as`)에 맞춰 배열 필드 형태로 결합되므로 단일 객체 매핑 시 후처리가 연이어 필요합니다.
- 조인 대상 컬렉션 (`from`)의 매칭 필드 (`foreignField`)에는 인덱스가 있는지 먼저 확인합니다.

```
db.users.aggregate([
  {
    $lookup: {
      from: "posts",
      // 조인 대상 컬렉션
      localField:
        "_id", // 기준 컬렉션
        매칭 필드
      foreignField:
        "user_id", // 대상 컬렉션
        참조 필드
      as:
        "user_posts"
      // 주입될 신규 배열 필드명
    }
  }
])
```

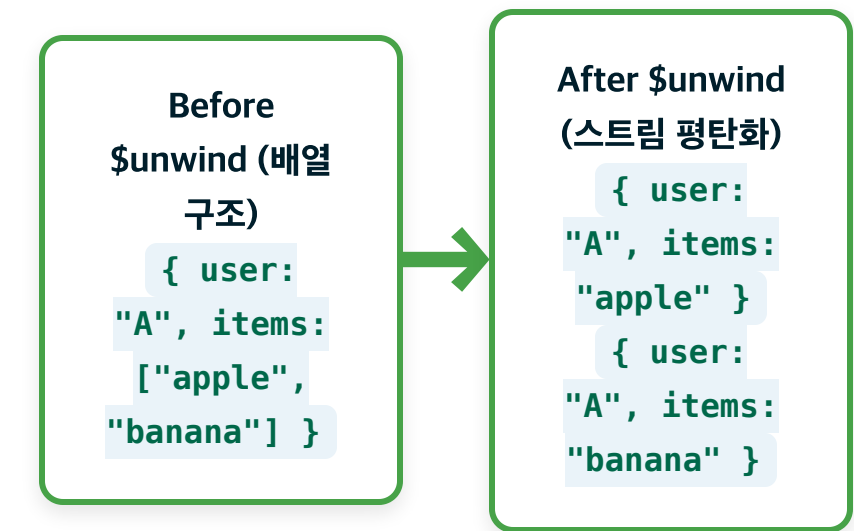
\$lookup 결과를 단일 객체처럼 다루기

- `$lookup` 결과는 배열로 들어옵니다.
- 1:1 참조는 `$unwind` 또는 `$arrayElemAt` 을 사용합니다.
- 단순 `$unwind:` `"$author"` 는 매칭된 작성자가 없을 때 해당 게시글을 결과에서 제외합니다.
- 조인 대상 컬렉션의 매칭 필드 인덱스를 확인합니다.

```
// MongoDB shell 예제
db.posts.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "author_id",
      foreignField: "_id",
      as: "author"
    }
  },
  {
    $unwind: {
      path: "$author",
      preserveNullAndEmptyArrays: true
    }
  },
  {
    $project: {
      title: 1,
      authorName: "$author.name",
      publishedAt: 1
    }
  }
]);
```

\$unwind 스테이지를 활용한 중첩 배열 해제

- 기능 설명: 단일 문서 내 배열 필드의 각 원소를 별도 문서처럼 펼쳐서 다음 스테이지로 전달합니다.
- 배열의 각 원소를 기준으로 그룹화하거나 정렬해야 할 때 사용합니다.



보안 및 API 변화에 대응하는 설계 원칙

Queryable Encryption (보안)

- 민감한 데이터 필드는 클라이언트 측에서 암호화되어 전송 및 보관됩니다.
- Equality와 Range 검색 조건은 이미 상용 환경에서 정식 지원 중입니다.
- 다만, 부분 문자열 (Substring) 등의 고급 질의 유형은 버전에 따라 public preview 단계에 속해 있으므로 도입 전 확인을 요합니다.

API 진화 대응 (App Services / Data API)

- 클라우드 종속 API는 지원 정책이 바뀔 수 있으므로 도입 전 현재 정책을 확인해야 합니다.
- 신규 시스템에서는 공식 드라이버와 백엔드 API 서버 기반 구조를 우선 검토합니다.

PART
6

실무 프로젝트: 대규모 블로그 플랫폼 설계

도메인 모델 수립부터 동시성 및 페이징 성능 극복
까지

블로그 시스템 요구사항 분석

- 시스템 분석 목표: 일일 수십만 뷰 이상 읽기 트래픽이 집중되며 다수 유저가 피드백(댓글, 좋아요)을 나누는 동적인 도메인 모델 설계입니다.
- 주요 엔티티와 관계 제약 사항:
 - 사용자(User): 회원 식별자, 닉네임, 권한(role) 및 기본 정보 프로필 보관
 - 게시글(Post): 제목, 내용, 저자 정보 참조, 게시 상태(draft, published)
 - 댓글(Comment): 단일 게시글에 대해 수만 개 이상 누적 가능. 무한 스크롤 기반 목록 페이지징 최적화 필수
 - 추천(Like): 동시 요청 시 중복 투표 방지 및 정확한 합산 카운팅 연계

읽기 성능 최적화를 위한 비정규화 설계

- 읽기 집중(Read-Heavy) 서비스 특징에 대응하기 위해, 조회 성능 강화 목적의 의도적 데이터 중복 비정규화를 설계합니다.
- 임베딩 방식 적용 예: 메인 피드 리스트 노출 시 매번 User 테이블 조인 조회를 거치지 않고, 글 작성 시점의 작성자명(username)과 썸네일 경로를 Post 문서에 직접 중복 저장해 둡니다.

갱신 오버헤드 주의:

저자가 닉네임을 변경하면 해당 작성자의 과거 작성글을 갱신하는 벌크 수정이 필요합니다. 따라서 데이터의 조회 빈도와 수정 주기를 먼저 확인해야 합니다.

과도한 참조 조인(Deep Join) 방지

- **안티패턴:** NoSQL 환경에서 정규화 편향에 의해 User, Post, Comment, Tag, Like 컬렉션을 개별로 나누고 조인을 수행하는 구조입니다.
- 단일 피드 노출 작업에서 작은 지연도 여러 번 누적되면 조인 비용이 커지고 쿼리를 관리하기 어려워집니다.
- **대안:** 게시글의 부가 정보 (예: 태그 리스트, 단순 좋아요 총량)는 Post 문서 안에 두어 한 번의 조회로 가져오도록 설계합니다.

문서 경계는 조회 단위에서 출발

MongoDB 모델링에서는 정규화 여부보다 먼저 “어떤 데이터를 한 번에 읽고 쓰는가”를 기준으로 문서의 범위를 정합니다.

PART 6 | 실무 프로젝트 대량의 댓글 처리를 위한 페이징 설계

- 용량 상한 이슈: 특정 인기 글에 댓글이 빠르게 늘어나는 경우, 문서 내에 `comments: []` 배열로 채우면 16MB 제한으로 서비스에러가 발생합니다.
- 처리 방식: 댓글 컬렉션을 분리하여 수십만 개의 데이터를 독립적으로 저장할 수 있게 합니다.
- 무한 스크롤에서는 `createdAt` 또는 `_id` 기반 커서 페이지네이션을 우선 검토합니다.

```
// 댓글 문서 분리 설계 예제
{
  "_id":
    ObjectId("60d5ec4b1234567890123456"),
  "post_id":
    ObjectId("60d5ec4b1234567890abcdef"),
  // 부모 게시물 식별
  "author": "Alice",
  "text": "해당 설계 원리에 동의합니다.",
  "createdAt": ISODate("2026-05-27T10:00:00Z")
}
```

PART 6 | 실무 프로젝트 댓글 목록 조회를 위한 인덱스

- 커서 페이징은 정렬 기준과 인덱스를 매칭해야 합니다.
- 동일 생성일(createdAt) 처리를 위해 `_id`를 보조 정렬 키로 활용합니다.
- 페이지가 뒤로 갈수록 `skip()`보다 커서 조건이 성능상 훨씬 유리합니다.

```
// MongoDB shell 예제
db.comments.createIndex({
  post_id: 1,
  createdAt: -1,
  _id: -1
});

db.comments.find({
  post_id: postId,
  $or: [
    { createdAt: { $lt: lastSeenCreatedAt } },
    {
      createdAt: lastSeenCreatedAt,
      _id: { $lt: lastSeenId }
    }
  ]
})
.sort({ createdAt: -1, _id: -1 })
.limit(20);
```

PART 6 | 실무 프로젝트 동시성 이슈를 고려한 추천(좋아요) 처리

- 경쟁 조건: 수십 명이 동시에 누를 때 단순 읽기-연산-저장(Read-Modify-Write)은 데이터 유실을 유발합니다.
- 원자성 확보: 조건절에 likes 필터링을 걸고 단일 업데이트 연산으로 원자성을 보장합니다.
- `likes: { $ne: userId }` 조건과 `$inc` 를 결합해 중복 차단 및 카운팅을 동시에 처리합니다.
- 좋아요 수가 계속 증가하는 서비스라면 별도 Like 컬렉션 분리를 검토합니다.

```
// 1. 추천(좋아요) 처리 - 원자적 조건 업데이트
const likeResult = await
Post.updateOne(
  { _id: postId, likes: { $ne:
userId } }, // 유저가 좋아요 안 누른
경우만 매칭
  {
    $addToSet: { likes: userId
},
    $inc: { like_count: 1 }
  }
);
// modifiedCount === 0 인 경우 이
미 누른 상태임

// 2. 추천 취소(좋아요 취소) 처리
const unlikeResult = await
Post.updateOne(
  { _id: postId, likes: userId
}, // 유저가 기존에 누른 상태인 경우만
매칭
  {
    $pull: { likes: userId },
    $inc: { like_count: -1 }
  }
);
```


PART 6 | 실무 프로젝트 좋아요가 커질 때: 별도 Like 컬렉션

- 좋아요 수가 매우 많다면 게시물 내 배열 대신 별도 컬렉션으로 분할 설계합니다.
- `post_id` 와 `user_id` 복합 unique 인덱스로 중복 좋아요를 방지합니다.
- 정확한 count가 중요하다면 트랜잭션을 사용하고, 약간의 지연이 허용되면 재집계나 비동기 보정 전략을 검토합니다.

```
// MongoDB shell 예제
db.likes.createIndex(
  { post_id: 1, user_id: 1 },
  { unique: true }
);

// ※ 두 작업의 원자성을 보장하려면 다중 문서
// 트랜잭션 필요
db.likes.insertOne({
  post_id: postId,
  user_id: userId,
  createdAt: new Date()
});

db.posts.updateOne(
  { _id: postId },
  { $inc: { like_count: 1 } }
);
```

계층형 댓글(대댓글) 데이터 모델링

- 인접 리스트 (Adjacency List): 직속 상위 댓글 ID(`parent_id`)를 기록합니다. 구조가 단순하나 무한 중첩 조회 시 N+1 쿼리 오버헤드가 발생할 수 있습니다.
- 조상 경로 배열 (Array of Ancestors): 상위 모든 조상 ID들을 `ancestors: []` 배열로 포함합니다. 단일 쿼리로 전체 하위 트리를 조회하기에 유리합니다.
- 구체화된 경로 (Materialized Paths): `"0001.0003.0012"` 와 같은 경로 문자열을 활용합니다. prefix 검색 인덱스로 하위 트리를 탐색합니다.
- 모델 선택 기준: 단순 댓글-대댓글(1단계)은 인접 리스트로 충분하며, 임의 깊이 트리 구조에는 조상 경로 배열이 권장됩니다.

핵심 요약 MongoDB 데이터 모델링 설계 수칙

1

조회 패턴 우선 분석

읽기/쓰기 비율과 단일 뷰에 필요한 필드를 분석합니다.

2

함께 읽히는 데이터는 임베딩 검토

조회 주기와 생명주기를 공유하는 데이터는 문서 내 임베딩을 우선합니다.

3

무한 성장 배열 방지

상한선 없이 자라는 배열은 16MB 초과를 막기 위해 참조 분리 설계합니다.

4

운영 인덱싱 및 보안 연계

복합 인덱스, Vector Search, 보안 (Encryption) 적용 여부를 조율합니다.

Q & A

질의응답 및 토론

ksl@jbnu.ac.kr

참고자료: MongoDB 공식 레퍼런스 가이드, Atlas Vector Search 기술 백서, Mongoose ODM 설계 가이드라인